

Project 2 (P2): Simplified $O(1)$ Scheduler

Instructor: Dr. Probir Roy

This project is on **xv6** scheduler. You are going to extend the **xv6** system you built in P1. There are three objectives to this assignment:

- To familiarize yourself with a real scheduler like **xv6** scheduler.
- To change that scheduler to a new algorithm.
- To display the scheduling information.

1 Simplified $O(1)$ Scheduler Design

The existing scheduler in **xv6** is a Round-Robin (RR) scheduler. On each timer interrupts (every 10 ms), the interrupt handler switches to the kernel scheduler, which then selects the next available process to run. This scheduler, while simple, is too primitive to do anything interesting. In this project, you will be putting a new scheduler – the simplified $O(1)$ scheduler – into **xv6**. $O(1)$ scheduler was used by Linux kernel.¹ The policy used by the $O(1)$ scheduler relies on active and expired queues of processes to achieve constant scheduling time. Each process in the active queue (AQ) is given a fixed time quantum 30 ms, after which it is preempted and moved to the expired queue (EQ). Once all the tasks from the active queue have exhausted their time quantum (in FIFO order) and have been moved to the expired queue, an queue switch takes place. Because the queues are accessed only via pointer, switching them is as fast as swapping two pointers. This switch makes the active queue the new empty expired queue, while the expired queue becomes the active queue whose time quantum will be reset as 30 ms.

In the original $O(1)$ scheduler in Linux, each process could be assigned with different priorities. For our simplified version, we assume all processes in the active queue have the same priority (low priority) except all newly created processes (high priority). Actually, we maintain another queue – called first-time queue (FQ) – which maintains all newly-created processes in the FCFS order. Once each new process exhausts its time quantum 10 ms, it will be put into the active queue. The purpose here is to give all newly-created processes the highest priority.

Therefore, after each timer interrupt is triggered, the scheduler first checks the FQ, if it is not empty, apply RR in the FQ; otherwise, apply RR in the AQ.

2 Testing

For the testing, in the **xv6** shell please run as follows:

```
$ spin 10000 &; spin 200000 &; spin 3000000 &;
```

¹Check this link for the details: <http://www.informit.com/articles/article.aspx?p=101760&seqNum=2>

These three spins are executed in a single command line and run concurrently in the background. Please use `cprintf()` in the kernel as you did in P1 to print on console the information for running `spin` processes as follows:

```
Process spin 12 has consumed 10 ms in FQ
Process spin 13 has consumed 10 ms in FQ
Process spin 14 has consumed 10 ms in FQ
Process spin 12 has consumed 10 ms in AQ
Process spin 12 has consumed 10 ms in AQ
Process spin 13 has consumed 10 ms in AQ
Process spin 13 has consumed 10 ms in AQ
Process spin 14 has consumed 10 ms in AQ
Process spin 14 has consumed 10 ms in AQ
...
```

Basically, at each timer interrupt, you print out the running process information. The information of the process ID and the process name, which are stored in the `proc` struct. Focus only the `spin` processes.

3 Tips

Most of the code for the scheduler is quite localized and can be found in `proc.c`; the associated header file, `proc.h` is also quite useful to examine. To change the scheduler, study its control flow and then try some small changes. For debugging the kernel functions, you could use `cprintf` to print to the console. You can find some examples in `proc.c`.

Since for every newly-created process `allocproc()` will be called, queue initialization should be done there. Figure out whether you need to clear up process when it is done. You can use a counter of timer interrupt to maintain the time quantum size of a process in a queue. For example, if the process is in Q_2 , then its initial counter will be 2. After each timer interrupt, it should be decreased by 1.

Resource The following two booklets are very helpful:

- Source code: <http://pdos.csail.mit.edu/6.828/2014/xv6/xv6-rev8.pdf>
It is a printed booklet with line numbers that keep all `xv6` source codes together.
- Commentary: <http://pdos.csail.mit.edu/6.828/2014/xv6/book-rev8.pdf>
It is written by the same team that ported `xv6` to x86. You can find in Chapter 5 in this booklet how the scheduler is designed.

4 Submission

You are going to report the following things:

- (1) Describe in details how you implemented the new scheduling policy in the `xv6`. A digram might be helpful;
- (2) Highlight all codes you have added or revised;
- (3) Write a detailed report with the screenshots of the testing results. Each screenshot should include your username and the current time, which show that you did it by yourself.

(4) Specify the contribution made by each member if you work as a group.

It should be written in a “.docx”, “.doc”, or “.pdf” format, and submitted to Project P2 on Canvas. Any compression file format such as .zip is not permitted.